Mercek — Azure SQL ile AI Search Otomatik Eşitleme

Bu doküman, Azure SQL'deki çözüm, talep ve duyuru kayıtlarının **AI Search'e kendiliğinden nasıl aktığını** anlatır: temel bilgiler, kurulum adımları, yetkiler, demo akışı ve sorun giderme.

Kod: `tools/mercek_data/search.py` (`setup_indexers`, `sync_now`), `load_all.py --search`, `db.py` (`search_text`, değişiklik takibi). Testler: `tools/mercek_data/tests/test_v7_v8.py`.

---

1. Temel bilgiler (basit anlatım)

- **Azure SQL defterdir:** bütün kayıtlar orada durur, uygulama oraya yazar. Tek doğruluk kaynağı SQL'dir.
- **AI Search akıllı kütüphanecidir:** "buna benzeyen hangisi?" sorusuna cevap verir. Bunun için metinlerin bir kopyasına ve o metinlerin anlamını taşıyan sayı listesine (embedding, vektör) ihtiyaç duyar.
- **Indexer otomatik kuryedir:** belirli aralıklarla SQL'e gider, "ne değişti?" diye sorar. Yeni, değişen ve silinen satırları alıp kütüphaneye taşır. Yolda her metni Foundry'nin `embedding` modeline gönderip vektörünü de ekler.

Sonuç olarak SQL'e yazılan bir çözüm ya da talep en geç **5 dakikada**, `--sync` ile tetiklenirse **birkaç saniyede** aranabilir hale gelir. Elle aktarma, iki yerin birbirini tutmaması ya da embedding'i ayrıca üretme gibi işler ortadan kalkar.

Hangi tablolar eşitlenir?

SQL tablosu	AI Search indeksi	Aranan metin (`search_text`)	Adet
`solutions`	`solutions`	başlık, belirti, kök neden, adımlar	37
`requests`	`requests`	başlık, açıklama	95 (+ demoda oluşan talepler)
`announcements`	`announcements`	kurum, başlık, metin	20

Loglar, müşteriler, krediler, temaslar ve çağrılar **eşitlenmez.** Onları saymak gerekir, benzerini aramak değil; SQL'de kalırlar.

Kullanılan Azure özellikleri

Özellik	Ne işe yarar
SQL değişiklik takibi (change tracking)	SQL hangi satırın eklendiğini, değiştiğini ya da silindiğini kendisi işaretler; indexer yalnızca onları okur
Hesaplanan kolon `search_text`	Aranacak metni SQL kendisi birleştirir; uygulama ayrıca yazmaz
AI Search veri kaynağı (azuresql)	"Şu veritabanının şu tablosuna bak" tanımı
Skillset + AzureOpenAIEmbeddingSkill (integrated vectorization)	Her metni Foundry `embedding` modeline gönderir, vektörü indekse yazar
Vektörleştirici (vectorizer)	Arama sırasında soruyu da Search kendisi vektöre çevirebilir
Indexer (zamanlanmış, 5 dk)	Hepsini birbirine bağlar ve çalıştırır

Akış

```
Uygulama / betik ──yazar──► Azure SQL (solutions · requests · announcements)
                                 │  değişiklik takibi: "şu satırlar değişti"
                                 ▼
                    AI Search indexer  (her 5 dk ya da --sync ile hemen)
                                 │  search_text ──► Foundry embedding ──► content_vector
                                 ▼
                    AI Search indeksleri ──► Mercek (benzer çözüm / talep / duyuru)
```

---

2. Kurulan nesneler (betik oluşturur)

Nesne	Ad	Not
Veri kaynağı × 3	`mercek-sql-solutions`, `mercek-sql-requests`, `mercek-sql-announcements`	`SqlIntegratedChangeTrackingPolicy` : silinenler de indeksten silinir
Skillset × 1	`mercek-embedding`	Girdi `/document/search_text` , çıktı `content_vector` , boyut `EMBEDDING_DIM`
İndeks × 3	`solutions`, `requests`, `announcements`	Türkçe analizör, HNSW (cosine), vektörleştirici `mercek-aoai` ; `requests.status` filtrelenebilir
Indexer × 3	`mercek-ix-solutions` , `mercek-ix-requests` , `mercek-ix-announcements`	Zamanlama `PT5M` , alan eşlemeleri (`solution_id → id` , `components_json → components` …)

Tümü `python3 tools/mercek_data/load_all.py --search` ile kurulur ve ilk çalıştırma beklenir. Komut her çalıştığında aynı tanımları yeniden yazar.

3. Yapılacaklar (hackathonda, sırayla)

Sorumlu: [6] Katalog ve senaryo verisi ile [14] AI Search (iş planı §2).

#	Adım	Nerede	Kontrol
1	AI Search servisinde **System-assigned Managed Identity**'yi aç	Portal → Search → Identity	Durum "On"
2	SQL sunucusunda **Microsoft Entra admin** tanımlı olsun	Portal → SQL server → Microsoft Entra ID	Admin atanmış
3	SQL'de Search'ün kimliğine okuma ve değişiklik takibi izni ver (aşağıdaki SQL)	Portal → SQL veritabanı → Query editor (Entra admin ile)	Hata yok
4	Foundry'de Search'ün kimliğine **Cognitive Services OpenAI User** rolü ver	Portal → Foundry kaynağı → Access control (IAM)	Rol listede
5	SQL güvenlik duvarında **"Allow Azure services and resources to access this server"** açık	Portal → SQL server → Networking	Açık
6	Ortam değişkenlerini tanımla (aşağıda)	Terminal (VDI)	`echo $SQL_RESOURCE_ID` dolu
7	`load_all.py --sql` → `load_all.py --search`	Terminal	Indexer'lar `success` ; 37 / 95 / 20
8	`validate_scenarios.py --sql --search`	Terminal	"SQL ↔ AI Search eşitliği" ✓
9	Backend: talep ya da çözüm yazan endpoint'ten sonra `POST /indexers/{ad}/run` çağrısı (isteğe bağlı)	`backend/app/clients/search.py`	Yeni kayıt saniyeler içinde aranabilir
10	Backend: benzer talep ve efor aramasında `filter = status eq 'tamamlandi'`	M2.2 talep analizi	D'nin TLP-1043'ü E'nin benzer taleplerinde çıkmaz

Adım 3 için SQL (`<search-adi>` = AI Search servisinin adı):

```
CREATE USER [<search-adi>] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [<search-adi>];
GRANT VIEW CHANGE TRACKING ON SCHEMA::dbo TO [<search-adi>];
```

Adım 6 için ortam değişkenleri (değerler repoya yazılmaz):

```
export SQL_SERVER=<sunucu>.database.windows.net SQL_DATABASE=<veritabani>
export
```

```
SQL_RESOURCE_ID=/subscriptions/<abonelik>/resourceGroups/<grup>/providers/Microsoft.Sql/servers/<sunucu>
export SEARCH_ENDPOINT=https://<search-adi>.search.windows.net
export AZURE_OPENAI_ENDPOINT=https://<foundry>.openai.azure.com
export EMBEDDING_DEPLOYMENT=embedding EMBEDDING_MODEL=text-embedding-3-small EMBEDDING_DIM=1536
```

`EMBEDDING_MODEL` ve `EMBEDDING_DIM`, Foundry'deki `embedding` deploy'unun gerçek modeliyle aynı olmalıdır.

Yetki çıkmazsa (yedek yollar)

Durum	Ne yapılır
Rol atama yetkisi yok (adım 1–4 yapılamıyor)	Kimlik yerine anahtar: `SEARCH_SQL_CONNECTION` (SQL kullanıcı adı ve şifresiyle tam bağlantı metni) ve `AZURE_OPENAI_API_KEY` ortamda tanımlanır. Betik bunları **yalnızca Azure'daki tanımlara** yazar; repoya, loga ya da koda girmez
Indexer hiç kurulamıyor	`load_all.py --search --search-mode push` : betik CSV'leri okur, embedding'i kendisi üretir ve belgeleri yükler. Otomatik eşitleme olmaz; yeni kayıt için komut tekrar çalıştırılır

4. Demo sırasında

An	Ne olur	Ne yapılır
Kurulum (1. gün)	`load_all.py --sql` tabloları yeniden yaratır ve değişiklik takibini açar; `--search` indexer'ları sıfırlayıp baştan çalıştırır	—
Her prova öncesi	`demo_reset.py --sql --sync-search` → TLP-1043 ve sonrası SQL'den silinir; indexer onları indeksten de siler	Komutun çıktısında indexer'ların tetiklendiği görünür
D (Fatma'nın talebi)	TLP-1043 SQL'e `planli` durumuyla yazılır; indexer onu `requests` indeksine taşır	E'de benzer talep aramasında `status eq 'tamamlandi'` filtresi olduğu için çıkmaz
Jüriye anlatım	"SQL'e yazılan her çözüm ve talep birkaç dakika içinde kendiliğinden RAG'e girer; embedding'i AI Search, Foundry ile kendisi üretir"	Gerekirse portalda indexer geçmişi gösterilir

5. Sorun giderme

Belirti	Olası neden	Çözüm
Indexer `persistentFailure` : "Change tracking is not enabled"	Tablolar yeniden yaratıldı, takip kapandı	`load_all.py --sql` tekrar (takibi kendisi açar), sonra `--search`
`Login failed for user '<token-identified principal>'`	Adım 3 yapılmadı ya da Search kimliği kapalı	Adım 1 ve 3
Skillset hatası 401/403	Foundry rolü yok ya da anahtar yanlış	Adım 4 veya `AZURE_OPENAI_API_KEY`
"Vector dimension mismatch"	`EMBEDDING_DIM` modelle uyuşmuyor	Deploy'daki modele göre düzelt, `--search` tekrar
İndekste yeni kayıt yok	5 dakika dolmadı	`load_all.py --sync`
`validate_scenarios` "SQL ↔ AI Search" ✗	Indexer henüz bitmedi ya da hata verdi	Portal → Search → Indexers → son çalıştırma ayrıntısı
SQL bağlantı zaman aşımı	Güvenlik duvarı	Adım 5

6. Maliyet ve sınırlar

- Indexer'ın kendisi ücretsizdir; yalnızca değişen kayıt için embedding çağrısı yapılır. İlk yüklemede 152 kayıt, sonrasında yalnızca yeni ya da değişen kayıtlar işlenir.

- En kısa zamanlama 5 dakikadır. Daha hızlısı için `run` çağrısı (`--sync`, `--sync-search` ya da backend'den) kullanılır.
- Değişiklik takibi saklama süresi 2 gündür. Indexer 2 günden uzun çalışmazsa kendini baştan yükler; hackathon için sorun değildir.